NeuroTechh: Architectural Decision Records

How to read these records

These 19 records explain why NeuroTechh is built the way it is, so each choice can be defended or revisited with a clear head. Every record uses the same six parts.

- **Context:** the problem and the forces on it.
- **Decision:** what we chose, in one or two sentences.
- **Alternatives:** what else was on the table, and why each lost.
- **Reasons:** why the choice won.
- **Consequences:** what we gain, and what it costs us.
- **Reconsider when:** the measurement or event that would reopen the decision.

Two principles run through all of them. The database decides correctness, so the app and API may be wrong or retried safely. And proven technology comes first: a new system is added only after a measured need.

Status. *Accepted* means it is the current design. *Proposed* means the direction is chosen but a detail, such as a number, still needs measuring.

ADR	Decision	Status
001	One modular API, not microservices	Accepted
002	Postgres is the single source of truth	Accepted
003	Dependencies point inward (ports and adapters)	Accepted
004	TLS and signed tokens, no service mesh	Accepted
005	A partial unique index guards every confirmed slot	Accepted
006	Row locks in a fixed order	Accepted
007	A pending request holds no slot	Accepted
008	Version-based optimistic concurrency on requests	Accepted
009	A reschedule is a new request that replaces the old appointment	Accepted
010	Sibling requests are declined by a retried job after commit	Accepted
011	No Redis for slot locking	Accepted

ADR	Decision	Status
012	Idempotency keys are created on the phone	Accepted
013	The phone saves an action before sending it	Accepted
014	Transactional outbox with a polling worker, no Kafka	Accepted
015	Retry only timeouts and 5xx	Proposed
016	One request ID serves as idempotency key and trace key	Accepted
017	Measure, hosted, for phone error tracing	Accepted
018	Telemetry carries IDs and outcomes only	Accepted
019	Audit log in Postgres, not ClickHouse	Accepted

Shape of the system

ADR-001: One modular API, not microservices

Context. The first brief asked for microservices. The core operation, confirming a slot for one patient, must check the slot and write the appointment so that two patients can never hold it.

Decision. Ship one deployable API with clear internal modules, backed by one Postgres database.

Alternatives.

- *Microservices (booking, scheduling, notification, identity):* each owns its data. Confirming a slot would span services, so it needs a distributed transaction or a saga with compensating steps.
- *Modular monolith with a later split:* chosen.

Reasons. Booking correctness is one local transaction. A saga adds states such as "half-booked" that must be detected and repaired. One deployable also means one thing to secure, deploy, monitor and debug.

Consequences. Simple to run and reason about. The cost: all modules scale together, and a bad deploy affects everything. Module boundaries (ADR-003) keep a future split possible.

Reconsider when. Separate teams are blocked on each other's releases, or one module has a measured scaling need the others do not share.

ADR-002: Postgres is the single source of truth

Context. Slots, requests, appointments, idempotency keys, the outbox and audit events must agree with each other at every moment.

Decision. All of them live in one Postgres database, and the application keeps no copy of the truth.

Alternatives.

- *Several stores, each best at one job* (a cache for slots, a log store for audit, a queue for events): each is fast at its job, but they can disagree.
- *One relational store:* chosen.

Reasons. Transactions across tables are free inside one database. Postgres is proven, widely understood and easy to hire for. We already run it.

Consequences. One backup, one restore, one access model. The cost: Postgres is the component we must protect hardest, with backups, a standby and restores that are practised, not assumed.

Reconsider when. A measured workload, after indexing, partitioning and a read replica, still cannot be served.

ADR-003: Dependencies point inward

Context. Booking rules must not depend on a web framework, a database driver or a vendor SDK, or each upgrade becomes a rule change.

Decision. The booking policy and its ports (interfaces) sit at the centre. Adapters for Postgres, push delivery, HTTP and telemetry sit outside and depend on the centre, never the reverse. On the phone this gives RequestStore, PendingQueue, SyncEngine, ApiClient and a Telemetry port with a Measure adapter and a Fake.

Alternatives.

- *Layered by technology* (controllers, services, repositories calling each other freely): quick to start, but rules and I/O drift together.
- *Ports and adapters:* chosen.

Reasons. The rules can be tested without a database or a network, and a vendor can be swapped by replacing one adapter.

Consequences. A few extra interfaces to write and keep honest. The gain is cheap tests and cheap replacement.

Reconsider when. A port has only ever had one adapter and no test double after a long time. Then it is ceremony and can be removed.

ADR-004: TLS and signed tokens, no service mesh

Context. Traffic must be encrypted and each caller identified.

Decision. TLS on every connection, signed tokens for identity, roles checked in the API.

Alternatives.

- *Service mesh with mutual TLS between services:* strong for many services, but it is a platform to run.
- *TLS and signed tokens:* chosen.

Reasons. With one API and one database there is little service-to-service traffic for a mesh to protect.

Consequences. Less to operate. The cost: if services multiply, identity between them has to be revisited.

Reconsider when. The API is split into several services that call each other.

Booking correctness

ADR-005: A partial unique index guards every confirmed slot

Context. Two clinic users, or two retries, can confirm different requests for the same slot at the same instant. Application checks alone can both pass.

Decision. A unique index on `appointment (slot_id) WHERE status IN ('CONFIRMED','COMPLETED')`. A violation is translated to SLOT_TAKEN.

Alternatives.

- *Check in code only:* fails under a race.
- *A unique index on all rows for the slot:* would block a new booking after a cancellation.
- *Partial unique index:* chosen, so cancelled appointments do not block the slot.

Reasons. The database refuses the second booking whatever the code did. It is the last line of defence and costs one index.

Consequences. Double booking is impossible by construction. The cost: code must handle the violation and turn it into a clean answer.

Reconsider when. A new appointment status can legitimately share a slot. Then the index condition must change with it.

ADR-006: Row locks in a fixed order

Context. A confirm touches the slot and the request. Two confirms that lock in opposite orders can wait on each other forever.

Decision. Lock the slot row first, then the request row, with `SELECT ... FOR UPDATE`. A reschedule locks both slots in ascending ID order.

Alternatives.

- *No locks, rely on the unique index:* correct, but concurrent attempts fail late and noisily.
- *Serializable isolation everywhere:* correct, but transactions can be aborted and must be retried broadly.
- *Explicit row locks in one order:* chosen.

Reasons. One lock order everywhere rules out deadlocks, and waiting callers see the final state and answer cleanly.

Consequences. Every new write path must follow the order, so it becomes a code-review rule. Locks are held only inside short transactions.

Reconsider when. Lock waits show up as a measured latency problem on hot slots.

ADR-007: A pending request holds no slot

Context. Patients ask; the clinic decides. If asking blocked the slot, one patient could keep it from everyone else.

Decision. A request in PENDING writes no hold on the slot. The slot becomes BOOKED only when a request is confirmed.

Alternatives.

- *Soft hold with a timeout:* friendlier to the requester but adds expiry timers, cleanup jobs and more states to get wrong.
- *No hold:* chosen.

Reasons. Fewer states, no timers, and the clinic stays in control of who gets the slot.

Consequences. Several patients may ask for the same slot, and all but one will be declined. The product must make that clear.

Reconsider when. Users complain of declined requests often enough that a short, visible hold would clearly help.

ADR-008: Version-based optimistic concurrency on requests

Context. A clinic user may confirm from a screen that is out of date, for example after the patient changed the request.

Decision. Each request has a version. A confirm carries the version the clinic saw, and a mismatch returns STALE_VERSION.

Alternatives.

- *Last write wins:* silently overwrites newer facts.
- *Lock the request while someone views it:* ties up rows for human think-time.
- *Version check:* chosen.

Reasons. No locks held while a person decides. The clinic reloads and decides again.

Consequences. One more error the app must explain well. The gain is that no decision is made on old data.

Reconsider when. STALE_VERSION appears often in normal use, which would point to a screen refresh problem.

ADR-009: A reschedule is a new request that replaces the old appointment

Context. Moving an appointment must never leave the patient with nothing if the new time is refused.

Decision. A reschedule is a new request carrying `replaces_appointment_id`. The old appointment is cancelled in the same commit that confirms the new one.

Alternatives.

- *Cancel first, then rebook:* a rejected rebook leaves the patient with no appointment.
- *Edit the appointment's slot in place:* hides history and complicates the unique index.
- *Replace on confirm:* chosen.

Reasons. The move succeeds as a whole or not at all, and both appointments stay in the history.

Consequences. The confirm path is longer, and it locks two slots (ADR-006).

Reconsider when. A new product rule changes who can move whose appointment.

ADR-010: Sibling requests are declined by a retried job after the commit

Context. When one request wins a slot, the other pending requests for it should be declined.

Decision. After the commit, a separate job declines the siblings. It is retried until it succeeds.

Alternatives.

- *Decline them inside the confirm transaction:* the transaction grows with the number of siblings and holds locks longer.
- *Leave them pending:* clinics would see requests that can never succeed.
- *Decline after the commit:* chosen.

Reasons. The booking commits fast. A sibling that is still pending for a moment is harmless, because confirming it fails with SLOT_TAKEN.

Consequences. A short window where stale pending requests are visible. The correctness rule is unaffected.

Reconsider when. The delay between booking and sibling decline is long enough for users to notice.

ADR-011: No Redis for slot locking

Context. A cache with locks is a common way to keep slots from being taken twice.

Decision. Do not use Redis, or any second system, for slot locks.

Alternatives.

- *Redis locks in front of Postgres:* fast, but the lock can expire or fail over and disagree with the database.
- *Postgres row locks and the unique index (ADR-005, ADR-006):* chosen.

Reasons. The lock and the data live together, so they cannot disagree. There is one fewer system to run, secure and learn.

Consequences. Hot-slot contention is handled by Postgres. We accept that rather than adding a component.

Reconsider when. Measured lock contention on Postgres limits throughput after query and transaction tuning.

Reliability on a bad network

ADR-012: Idempotency keys are created on the phone

Context. A response can be lost after the server has already committed. The phone then cannot tell whether the booking happened.

Decision. The phone creates a key K for each action, saves it with the action, and sends it as `Idempotency-Key`. The API stores the result with a hash of the payload. A replay returns the stored result. The same key with a different payload returns 409 IDEMPOTENCY_CONFLICT.

Alternatives.

- *Server-generated keys:* the server cannot know that a retry is the same action unless the client says so.
- *Natural keys* (patient, slot, time): block legitimate repeats and fail on reschedules.
- *Client-generated key:* chosen.

Reasons. Only the client knows whether it is retrying or acting anew. A stored result makes a retry safe.

Consequences. Idempotency records must be stored and eventually cleaned up. How long to keep them is a number to set from real retry behaviour, and we do not assume one here.

Reconsider when. Another client type, such as a partner integration, cannot generate keys reliably.

ADR-013: The phone saves an action before sending it

Context. The app can be killed, or the network can drop, between a tap and the reply.

Decision. The action and its key are written to local storage first (PendingQueue). Only after the API's answer is the saved action cleared.

Alternatives.

- *Send first, keep it in memory:* a lost action is lost silently.
- *Save, then send:* chosen.

Reasons. The saved action survives restarts, and together with ADR-012 a retry cannot double-book.

Consequences. The app needs local storage and a sync engine, and the UI must show "waiting to send" honestly.

Reconsider when. Local storage problems, such as corruption or growth, appear in the field.

ADR-014: Transactional outbox with a polling worker, no Kafka

Context. Confirming a booking must notify the patient, but a notification must never be sent for a booking that rolled back, nor lost for one that committed.

Decision. An outbox row is written in the same transaction as the change. A worker polls the table after the commit and delivers, retrying on failure.

Alternatives.

- *Call the push service inside the transaction:* a slow or failing vendor holds locks, and a rollback can still leave a sent message.
- *Publish to Kafka from the app after commit:* the process can die between commit and publish, and the event is lost.
- *Kafka with change-data-capture:* correct, but adds a broker and connectors to run.
- *Outbox plus polling worker:* chosen.

Reasons. One transaction writes both the change and the event, so they cannot disagree. A polling worker is the simplest thing that works and uses what we already run.

Consequences. Delivery is at-least-once, so consumers must tolerate duplicates. Polling adds a small delay, and the table needs routine cleanup.

Reconsider when. Notification delay or table size is measured as a problem, or several independent consumers need the same events.

ADR-015: Retry only timeouts and 5xx

Context. Retrying everything wastes effort and can hide real errors. Retrying nothing loses actions on bad networks.

Decision. Retry network timeouts and 5xx with growing delays, up to a limit. 409 and 422 are final answers and are never retried. The limit and delays are proposals to be set after measuring (a first guess is at most five attempts).

Alternatives.

- *Retry every failure:* repeats rule violations that cannot change.
- *No automatic retry:* the user must retry by hand on every bad signal.
- *Selective retry:* chosen.

Reasons. A 409 or 422 comes from the rules, so the same request gets the same answer. A timeout or 5xx may succeed later, and ADR-012 makes the repeat safe.

Consequences. The app must show a clear message for final errors and a quiet "still trying" for retryable ones.

Reconsider when. Telemetry (ADR-017) shows retries that never succeed or that overload the API.

Status. Proposed, until the attempt count and delays are measured.

Observability, privacy and audit

ADR-016: One request ID is the idempotency key and the trace key

Context. When a booking goes wrong, someone must follow it from the phone to the database. Two separate IDs would need a lookup between them.

Decision. The request ID K is sent as `Idempotency-Key` and `X-Request-Id`. It appears in API log lines and on the outbox row, and is a Measure attribute on the phone. The outcome is one of: ok, network_timeout, rejected_slot_taken, rejected_validation, server_error.

Alternatives.

- *Separate trace ID:* one more thing to generate and join.
- *Full distributed tracing platform:* powerful, but it is another system to run for a single API.
- *Reuse K:* chosen.

Reasons. One ID answers "what happened to this tap" in every place it touched.

Consequences. The key must never carry meaning or personal data. It is a random identifier only.

Reconsider when. Several services call each other, so one action needs a tree of spans.

ADR-017: Measure, hosted, for phone error tracing

Context. Errors that happen on the device, such as a timeout or a crash, never reach the API logs.

Decision. Use Measure from the phone app, behind a Telemetry port with a Measure adapter and a Fake for tests. Use the hosted service instead of running our own.

Alternatives.

- *Build our own error pipeline:* more control, much more to run.
- *Self-host Measure:* Measure's docs recommend a single Linux machine with at least 4 vCPU, 16 GB and 100 GB, and 32 GB or more for its ClickHouse. That adds a database we would operate, which cuts against our own technology policy. The docs recommend the hosted cloud for scale.
- *Hosted Measure:* chosen.

Reasons. It gives crash and error tracing without a new database for us to run, and the port keeps it replaceable.

Consequences. Data leaves our systems, so ADR-018 limits what is sent. Open points still to verify before release: the React Native method signatures, Expo Go support, SDK overhead and the masking setting names.

Reconsider when. Price, privacy terms or reliability change, or the SDK cannot meet the open points above.

ADR-018: Telemetry carries IDs and outcomes only

Context. NeuroTechh handles health-adjacent appointment data, and monitoring tools are a common way for it to leak.

Decision. Allowed: IDs, attempt number, outcome, HTTP status, screen and role. Forbidden: names, contact details, notes, free text and slot times.

Alternatives.

- *Send everything and mask later:* one missed rule leaks data.
- *Allow-list of fields:* chosen.

Reasons. An allow-list fails closed: a new field is blocked until someone adds it on purpose.

Consequences. Debugging sometimes needs a lookup by ID in our own database, because the monitor shows no details.

Reconsider when. A proposed new field has a clear, reviewed need. It goes through review, not through a quiet code change.

ADR-019: Audit log in Postgres, not ClickHouse

Context. We need a tamper-evident record of who did what. The first idea was ClickHouse, which would be a new database added for one purpose.

Decision. An `audit_event` table in the Postgres we already run. The writer role has INSERT only and the reader role SELECT only. The table is partitioned by month, with a hash chain across rows and a nightly export to write-once storage. An analytics copy is added only if a real reporting need appears.

Alternatives.

- *ClickHouse:* strong for large analytical scans, but audit is mostly writes and lookups by ID. It adds a database to run, secure, back up and hire for, and the audit write would be a second write that can fail.
- *Application log files:* easy, but hard to query and easy to alter.
- *Postgres with the controls above:* chosen.

Reasons. The audit row is written in the same transaction as the booking, so both exist or neither does. We already operate Postgres. The controls make silent edits detectable.

Consequences. The audit table grows with usage and needs partition management and archiving. A very heavy reporting query could compete with live traffic, so reports run on a replica.

Reconsider when. Measured audit volume or reporting scans cannot be served by partitioning, indexes and a replica. The analytics copy is the first step, before any move of the source of truth.

Walkthrough 1: the core path

A patient requests a slot, the clinic confirms it, and the patient is told. This is the path every other record protects.

Expected behavior

Step	What happens	What is written	What the user sees
1. Patient taps Request	The app saves the action with a new ID K, then sends it with `Idempotency-Key: K` (ADR-012, 013).	A local saved action on the phone.	"Waiting to send".
2. API receives it	Checks the token, looks up K, opens a transaction, locks the slot row, confirms the slot is OPEN (ADR-006).	Nothing yet.	Nothing yet.
3. Commit	One commit writes the PENDING request, an outbox row and the stored result for K (ADR-007, 014).	Request, outbox row, idempotency result.	201 returned; the saved action is cleared; the request shows as pending.
4. Clinic opens the request	Reads the request with its current version.	Nothing.	The request in the clinic's list.

Step	What happens	What is written	What the user sees
5. Clinic confirms	Locks slot then request, checks PENDING, version and slot OPEN, then commits (ADR-005, 006, 008).	Appointment CONFIRMED, request CONFIRMED, slot BOOKED, outbox row, audit event.	Confirmation on the clinic screen.
6. After the commit	The worker delivers the outbox row; a job declines sibling requests (ADR-010, 014).	Delivery state on the outbox row; siblings set to DECLINED.	The patient is told; other patients see a decline.

Things that must always be true.

- A slot has at most one CONFIRMED or COMPLETED appointment.
- A pending request never changes the slot.
- Every committed change has its outbox row, and every outbox row belongs to a committed change.
- A rejection writes nothing.

Verification

Check	How	Expected
Happy path	Integration test: request, confirm, read back.	Appointment CONFIRMED, slot BOOKED, one outbox row per change.
No double booking	Test that confirms two requests for one slot from two connections at once.	One succeeds; the other gets SLOT_TAKEN.
Index is the last guard	Test that bypasses the code check and inserts a second CONFIRMED appointment for the slot directly.	The database raises a unique violation.
Replay	Send the same command twice with the same K.	Same answer; one appointment.
Key reuse	Send the same K with a different payload.	409 IDEMPOTENCY_CONFLICT.

Check	How	Expected
Live invariant	Query for slots with more than one CONFIRMED or COMPLETED appointment: `SELECT slot_id FROM appointment WHERE status IN ('CONFIRMED','COMPLETED') GROUP BY slot_id HAVING count(*) > 1`.	Zero rows, always.

Diagnostics. Start from the request ID K. Search Measure for K on the phone, then the API log for the same K in `X-Request-Id`, then the outbox row carrying K (ADR-016). The `outcome` on the phone says where it stopped: ok, network_timeout, rejected_slot_taken, rejected_validation or server_error. If the phone says ok but the patient was not told, look at the outbox row: not delivered means the worker, delivered means the push service or the device.

Walkthrough 2: when things fail

Three failures, each with the behavior we expect, how we prove it, and how we find it in production. In all three, the system ends in a correct state without anyone repairing data by hand.

Failure A: the response is lost after the server committed

What happens. The patient taps Request. The API commits, but the phone loses signal before the 201 arrives. The app sees a timeout.

Expected behavior.

1. The phone records the outcome `network_timeout` and keeps the saved action, because it was never cleared (ADR-013).
2. It retries with the same K after a growing delay (ADR-015). The API finds K, sees the same payload hash, and returns the stored 201.
3. The phone clears the saved action. There is one request in the database, not two.

Verification. Test with a proxy that drops the response after the API commits. Assert one request row, one outbox row, and a second call that returns the first answer. A second test sends the same K with a changed payload and expects 409 IDEMPOTENCY_CONFLICT, which proves a replay is not a blind cache.

Diagnostics. In Measure, K shows one attempt with `network_timeout` followed by an attempt with `ok`. The API log shows two lines for K, the second marked as a replay. If the second attempt never appears, the phone never retried: check whether the saved action

still exists on the device.

Failure B: two clinic users confirm different requests for the same slot

What happens. Two staff members each confirm a different patient for the same slot at the same moment.

Expected behavior.

1. Both transactions try to lock the slot row. One gets it first; the other waits (ADR-006).
2. The first commits: appointment CONFIRMED, slot BOOKED.
3. The second gets the lock, sees the slot is no longer OPEN, and returns 409 SLOT_TAKEN. Nothing is written.
4. If both ever passed the code checks together, the unique index makes the second insert fail, and the API still answers SLOT_TAKEN (ADR-005).
5. The second clinic user sees a clear message and the slot disappears from their open list.

Verification. The concurrent-confirm test from Walkthrough 1, repeated many times. Expected every time: exactly one CONFIRMED appointment for the slot and one SLOT_TAKEN. A separate test removes the code check on purpose and expects the unique violation, to prove the index works on its own.

Diagnostics. Two API log lines with different K values for one slot ID, one ok and one `rejected_slot_taken`. A `rejected_slot_taken` is normal when staff collide. A sudden rise in them points to a screen that is not refreshing, which is a product issue and not a data issue.

Failure C: the notification cannot be delivered

What happens. The booking is committed but the push service is down, or the worker crashes.

Expected behavior.

1. The booking stays correct. The outbox row is already committed (ADR-014).
2. The worker retries the row until it is delivered. After the service returns, the patient is told, late but told.
3. Because delivery is at-least-once, a crash between sending and marking as delivered can send twice. The message is safe to repeat.

Verification. Stop the push adapter in a test, confirm a booking, and assert the outbox row remains undelivered. Restart the adapter and assert delivery. Kill the worker between send and mark, restart, and assert the message is repeated safely and the row ends delivered.

Diagnostics. Look for outbox rows that stay undelivered for longer than the delay you can accept (set from the product, not guessed here). Many old rows means the worker or the push service; a few means one bad destination, such as an invalid device token. Alert on the age of the oldest undelivered row.

Walkthrough 3: a new requirement arrives

The requirement. Clinics ask: "Let us hold a slot for a patient while they fill in an intake form." The length of the hold is set by the clinic. This is realistic, and it directly challenges ADR-007, which says nothing holds a slot until it is confirmed.

Expected behavior. A clinic user can hold an open slot for one named request. While the hold is active, other requests cannot be confirmed for that slot, and the held request can. When the hold ends or is released, the slot is open to everyone again.

How the design absorbs it. The answer is a new record, not a rewrite.

- **ADR-020 (Proposed): clinic-initiated holds.** It narrows ADR-007: asking still holds nothing, but a clinic can create an explicit hold. ADR-007 stays valid for ordinary requests.
- **A hold is data on the slot**, not a new kind of appointment: which request it is held for and until when. The slot's status stays OPEN or BOOKED.
- **Expiry is decided by the clock at check time.** A hold whose end time has passed is treated as gone by every read and by the confirm path. A cleanup job only tidies old values, so a late job cannot keep a slot blocked.
- **The confirm path gains one check**, made after the slot is locked: if there is an active hold for a different request, return a new final error SLOT_HELD.
- **Hold and release are ordinary commands**: idempotency key, slot lock first, one commit, an outbox row and an audit event. Only the clinic role may call them.

Area	Changes?	How
Unique index (ADR-005)	No	A hold is not an appointment, so the guard is untouched.
Lock order (ADR-006)	No	Hold and release lock the slot first, like everything else.
Pending holds no slot (ADR-007)	Narrowed	Superseded for held requests only, by ADR-020.
Idempotency and retries (ADR-012, 015)	No	New commands use the same keys; SLOT_HELD is final and not retried.

Area	Changes?	How
Outbox (ADR-014)	Extended	A new event type tells the patient the slot is held for them.
Telemetry (ADR-016, 018)	Extended	A new outcome, `rejected_slot_held`. The allowed fields do not change.
Audit (ADR-019)	Extended	Hold and release are audit events.
Database	Additive	Two nullable columns on the slot. Existing rows and code keep working.
Phone app	Extended	A "held for you" state and a clear SLOT_HELD message for other clinic users.

Verification.

Check	Expected
Hold a slot, then confirm a different request.	SLOT_HELD, nothing written.
Hold a slot, then confirm the held request.	Confirmed, the hold is cleared in the same commit.
Let the hold pass its end time, then confirm a different request.	Confirmed, with no cleanup job needed.
Race: confirm and release at the same instant.	One order wins; the slot is never double booked, and the unique index still holds.
Retry the hold command with the same K.	One hold, the same answer.
Run the whole existing test suite.	Unchanged results, which proves the change is additive.

Diagnostics. Support will ask "why can't I confirm this?" The answer is `rejected_slot_held` in Measure and a SLOT_HELD line in the API log, with the hold's request ID and end time in the audit event. A hold that never ends points to a bad end time written by the hold command, not to a cleanup job, because expiry does not depend on one.

Risks, and what we do.

- *A clinic holds many slots and starves patients.* Only the clinic role can hold, every hold is audited, and the clinic sets its own length. A per-clinic limit can be added if misuse is measured.

- *Clock differences.* Hold times are compared using the database's clock, not the phone's.

Reconsider when. Holds become a core flow rather than an exception, for example patients holding their own slots. Then the hold deserves its own table and states, and ADR-007 should be replaced rather than narrowed.